CAPSTONE PROJECT | WEB APPLICATION SECURITY

Web Application Penetration Testing and Incident Response Using DVWA

A practical, evidence-driven SQL injection assessment in a controlled training environment

Prepared by: [FILL IN NAME]

Program / internship: [FILL IN]

Assessment date: [FILL IN]

Target: DVWA local lab at [VERIFY URL]

IMPORTANT - DRAFT TEMPLATE. This is not a completed penetration-test result. Actual SQLi behavior, screenshots, logs, and response outcomes must be added from the user's own lab. No vulnerability is claimed as confirmed in this document.

Primary deliverables

Report PDF | GitHub-ready repository | Network diagram | Evidence register | Offline log detector | 12-minute video script

Method reference: OWASP Web Security Testing Guide and SQL Injection Prevention Cheat Sheet

1. Executive Summary

This capstone is designed to test a deliberately vulnerable DVWA instance, document a controlled SQL injection experiment, and simulate basic incident detection and response using local web logs. It emphasizes scope control, observable evidence, practical mitigations, and honest reporting.

Current report status: Draft / evidence pending. The supplied browser screenshot shows the DVWA home page at a loopback URL, but it does not establish that a SQL injection test was performed. Replace the placeholders below only after testing.

Actual SQLi result	[CONFIRMED / NOT CONFIRMED / INCONCLUSIVE - fill after test]
Target URL	[Verify current live endpoint, e.g. http://127.0.0.1:4280]
Security setting	[Record selected DVWA security level]
Evidence	[E01-E06 filenames in evidence register]
Retest status	[Not performed / performed with results]

2. Objectives and Scope

Objectives

- Establish normal baseline behavior of the DVWA SQL Injection module.
- Use a limited input-validation and Boolean-condition test in the local training target.
- Record screenshots and sanitized HTTP/log evidence, then explain actual results.
- Recommend secure query handling and demonstrate basic log triage/incident response.

Scope

- In scope: the project owner's local DVWA instance, initial security level Low, and logs generated by the controlled test.
- Out of scope: public sites, unrelated systems, destructive changes, denial of service, credential/hash dumping, and real user data.

Authorization and safety

All testing is limited to a deliberately vulnerable local lab. Keep DVWA bound to loopback or an isolated lab network. Do not place the app on a public interface. Save only sanitized evidence.

3. Environment and Architecture

Record the real environment below. Do not assume that a browser tab or cached page means the service is currently running.

Kali VM / OS	[Distribution and version]
DVWA version / image	[FILL IN]
URL and port	[Verify live URL]
Runtime	[Docker Compose / other]
Log source	[Apache access log / container logs / other]

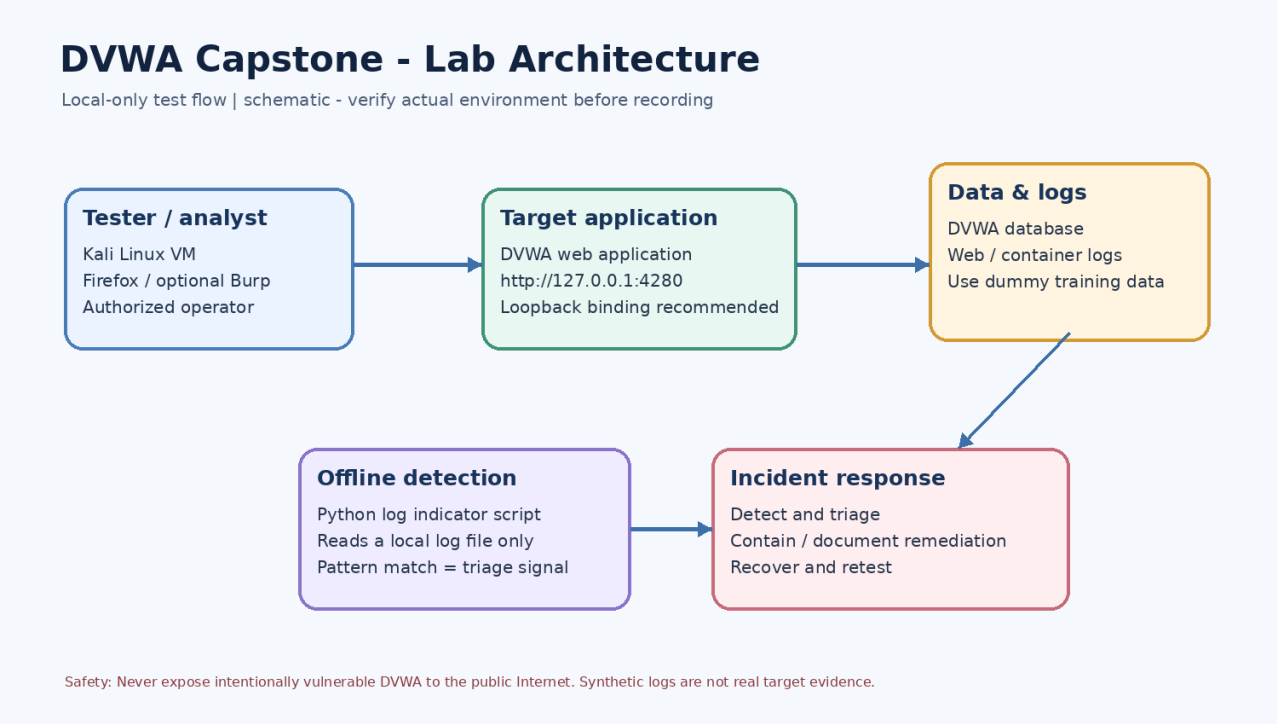


Figure 1. Intended local architecture. Confirm actual interfaces and ports during setup; the diagram is a plan, not a discovery result.

Initial application screenshot

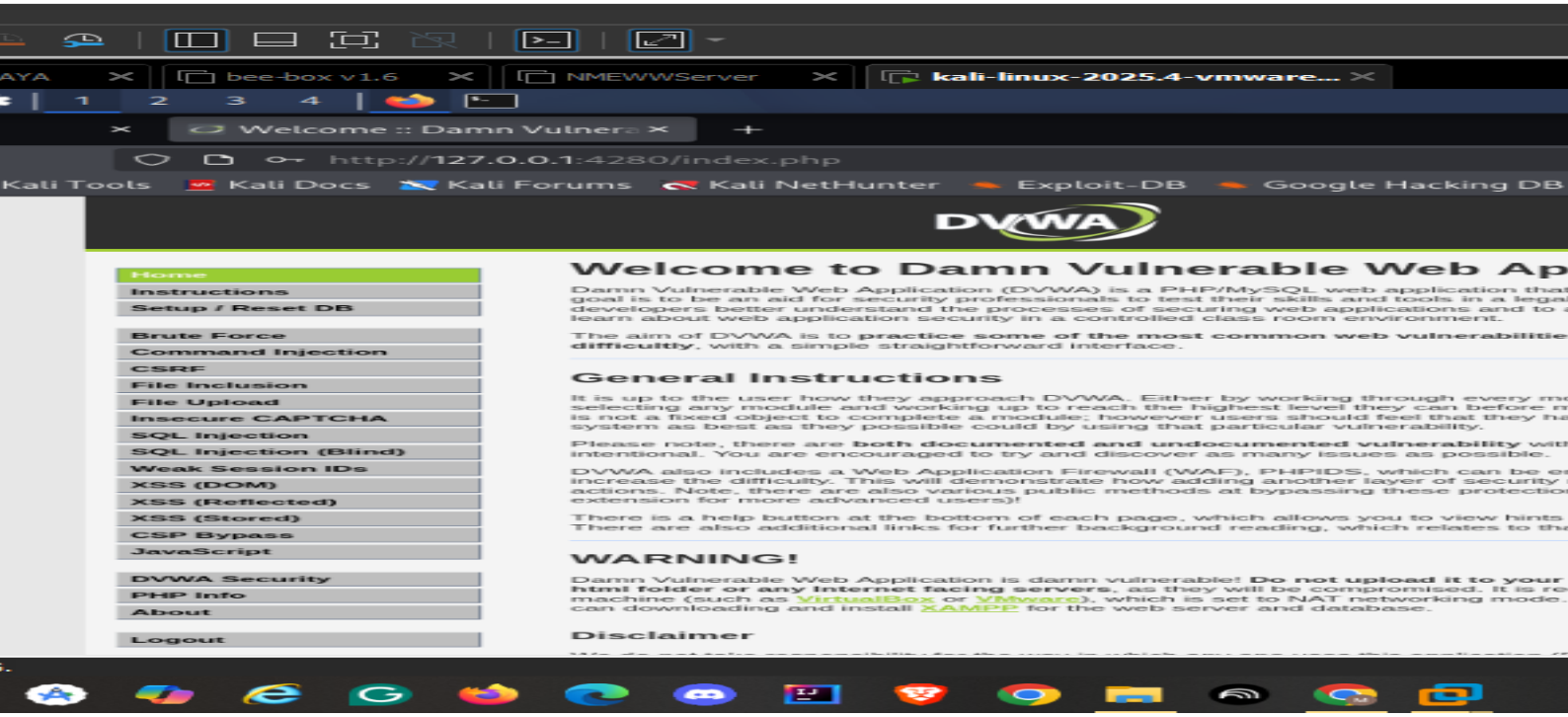


Figure 2. User-provided DVWA home page screenshot with a loopback URL. It is setup evidence only and does not demonstrate a SQLi result.

4. Methodology and Controlled Test Procedure

Use a repeatable sequence: confirm scope and isolation, verify only the local DVWA port with curl/Nmap, capture a normal baseline, make one controlled input change at a time, compare responses, preserve evidence, and stop after the demonstrated effect.

1. Recon / scan	Verify only 127.0.0.1:4280 using curl and a narrow Nmap service check; record the actual output.	E01
2. Baseline	Open SQL Injection; submit `1`; record the normal output.	E02
3. Syntax probe	Submit `1'`; record error, rejection, or normal response exactly.	E03
4. Boolean comparison	Compare `1' OR '1'='1` with `1' AND '1'='2`.	E04
5. Log review	Inspect local web logs around the test time and run the offline indicator scanner.	E05
6. Retest	Record mitigation advice and retest only if a fix was actually applied.	E06

Important difference between the supplied tutorial and this project

The supplied video transcript discusses bWAPP's search input and GET parameter. This project uses DVWA, where the standard SQL Injection module is commonly a User ID input. Do not copy bWAPP parameter names or assume that its selected-column count matches DVWA.

Controlled SQLi input examples

```
Baseline: 1
Quote probe: 1'
True condition: 1' OR '1'='1
False condition: 1' AND '1'='2
```

Use these only in your own DVWA instance. Compare what your page actually shows. Do not dump password hashes, enumerate unrelated tables, alter records, or run destructive queries.

What counts as evidence

- A baseline screenshot and the exact security level.
- A screenshot of the quote probe response; do not infer more than it supports.
- A paired true/false response screenshot or equivalent Burp request/response with cookies redacted.
- The timestamp and a relevant, sanitized log line if available.

5. Finding and Mitigations

Finding F-01: SQL Injection test in DVWA

Status	[CONFIRMED / NOT CONFIRMED / INCONCLUSIVE]
Affected module/parameter	[FILL IN from the actual request]
Severity	[Rate with a rationale based on observed impact and context]
Evidence IDs	[E02, E03, E04; add actual file paths]
Finding summary	[Describe baseline versus test responses without exaggeration]

Technical explanation

SQL injection may occur when user-controlled input is concatenated into SQL syntax instead of being bound as data. The impact depends on the query, privileges, database configuration, and reachable data. Report only effects observed in this lab and separate them from theoretical worst-case impact.

Recommended mitigations

- Use parameterized queries / prepared statements; do not build SQL by concatenating input.
- Apply allow-list validation when the field is expected to be a known identifier or other restricted value.
- Use a database account with the minimum permissions required by the application.
- Return generic client-facing errors; keep detailed diagnostics in restricted server logs.
- Add regression tests for normal, invalid, and SQL-metacharacter inputs.
- Monitor suspicious request patterns; treat pattern matches as triage signals, not proof by themselves.

Retest and status

[If source code was not modified and the fix retested, say: "Mitigations recommended; application-level fix not applied or verified in this exercise." Do not call the finding remediated without evidence.]

Severity guidance

For this intentionally vulnerable training application, severity is a learning assessment. If confirmed, justify the rating using demonstrated data exposure or query manipulation, required access, and likely impact in a hypothetical production context. Do not call a finding Critical merely because the module is designed to be vulnerable.

6. Incident Detection and Response Simulation

Scenario: a controlled SQLi-like request is sent to local DVWA during the test window. The analyst reviews local logs, identifies the request, documents containment, and records remediation/retest status.

Detection script

The included `scripts/detect_sqli_logs.py` reads a local text file and flags a small set of suspicious SQL-related strings. It is offline: it does not generate requests. Results are heuristic and may include false positives and false negatives.

Example command

```
python3 scripts/detect_sqli_logs.py evidence/actual_access.log
```

The included `scripts/sample_access.log` is synthetic. If you use it to demo the detector, state on the video and in the report that it is synthetic, not output from your target.

Detect	Flag suspicious request pattern and timestamp.	[log source / output]
Triage	Correlate with the controlled test time and browser evidence.	[request / EIDs]
Contain	For this disposable lab, stop DVWA after preserving evidence or restrict local access.	[action + time]
Eradicate / remediate	Document prepared statements and least privilege; apply and verify only if in scope.	[change/retest]
Recover	Verify the expected service state and retest safely if a fix was made.	[result]
Lessons learned	Improve query handling, error handling, logging, monitoring, and regression testing.	[summary]

Post-incident summary to complete

Incident ID: IR-01
Start/end time: [FILL IN]
Detection source: [FILL IN]
Indicator: [FILL IN]
Containment performed: [FILL IN]
Remediation/retest result: [FILL IN]
Residual limitations: [FILL IN]

7. Deliverables and Evidence Checklist

PDF report	Executive summary, scope, methodology, findings, mitigations, incident response, references, actual screenshots.	Template - update
GitHub repository	README, checklist, diagram, script, evidence register, report source.	Starter kit
12-minute video	Live walkthrough plus actual outputs and honest limitations.	Script included
Final link list	Public/private repo URL, hosted report URL, video URL.	Fill after publishing

Evidence register (minimum set)

E01	E01_dvwa_home.png	Live DVWA URL and environment
E02	E02_sqli_baseline.png	Normal request/response
E03	E03_sqli_quote_test.png	Controlled quote test
E04	E04_sqli_boolean_test.png	True/false comparison
E05	E05_access_log_detection.png	Log source and detector output
E06	E06_remediation_retest.png	Retest; if no fix applied, show mitigation plan instead

Submission QA

- Replace every `[FILL IN]` marker; remove this warning only after evidence is complete.
- Make sure the finding status is supported by the screenshots and observed responses.
- Label the synthetic log sample as synthetic if used.
- Redact cookies, session IDs, tokens, and private personal data.
- Open the final PDF and play the video before sharing links.

8. References and Learning Links

Learning videos

1. bWAPP SQL injection tutorial supplied for background:
https://www.youtube.com/watch?v=ZaLWQ89b5f8
2. DVWA SQL Injection (low/medium/high/impossible) walkthrough:
https://www.youtube.com/watch?v=5bj1pFmyyBA
3. SQL Injection for Beginners (DVWA walkthrough):
https://www.youtube.com/watch?v=aYFZp80eOb0

Note: the first video transcript is about bWAPP and its search parameter. The lab here uses DVWA; test the DVWA field and request as it actually behaves. Do not claim you performed a bWAPP test.

Authoritative references

1. OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
2. OWASP SQL Injection testing guide: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Injection/05-SQL_Injection/
3. OWASP SQL Injection Prevention Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
4. Official DVWA repository: https://github.com/digininja/DVWA

Final note

This PDF is a professionally structured starter report, not a fabricated completed assessment. The actual test outcome and screenshots must come from the local DVWA run. Update the report and regenerate the PDF before final submission.

Report template ends here